Web Programming

Quiz Material

Eötvös Loránd University

2026-2027/1



Project structure

- `index.html`, which contains:

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <script src="script.js" defer></script>
</body>
```

- `style.css`
- `script.js`

First program: Hello World!

```
console.log("Hello World!");
```

When opening the webpage, the text appears in the browser console.

- In Chromium-based browsers: `Ctrl + Shift + J`
- In Firefox: `Ctrl + Shift + K`

Variables

```
let x = 5;     // block-scoped variable
const y = 10; // constant (must be initialized)

var z = 15;    // function-scoped/global
               // variable (DEPRECATED!)
```

Types and their literals

- **Primitive types:**
  - `Boolean`: `true`, `false`
  - `null`: variable without a value
  - `undefined`: uninitialized variable
  - `Number`: `5`, `3.14`, `NaN`, `Infinity`
  - `BigInt`: `9007199254740991n`
  - `String`: `"Hello World!"`, `'Hello World!'`, `""`, `` `Hello ${name}!` `` (where `name` is a `String` expression)
  - `Symbol`: unique identifier
- **Composite types:**
  - `object`: `{ key1: "value1", key2: "value2" }`
  - `array`: `[1, 2, 3, 4, 5]`, `[]`
  - `function`: `function myFunction() { return "Hello World!"; }`

Source: *MDN Web Docs*

Types and their operations

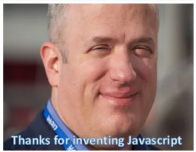
The operations belonging to the types are very similar to those in C-like languages:

- `Number` and `BigInt`: `+`, `-`, `*`, `/`, `%`, `**`, `++`, `--`
- `String`: `+` (concatenation)
- `Boolean`: `&&`, `||`, `!`
- Comparison operators: `==`, `!=`, `===`, `!==`, `<`, `>`, `<=`, `>=`, `instanceof`
- Assignment operators: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`

Source: *MDN Web Docs*

Type coercion

```
> typeof NaN
"number"
> 9999999999999999
10000000000000000
> 0.5+0.1==0.6
true
> 0.1+0.2==0.3
false
> Math.max()
-Infinity
> Math.min()
Infinity
> []+[]
""
> []+{}
"[object Object]"
> {}+[]
0
> true+true+true===3
true
> true-true
0
> true==1
true
> true===1
false
> (!+[]+[]+![]).length
9
> 9+"1"
"91"
> 91-"1"
90
> []==0
true
```



Source: *Reddit*

- + operator
  - `Number` + `Number` ⇒ addition
  - `Object` + _ ⇒ addition
  - otherwise ⇒ concatenation, and operands are converted to strings (`toString()` function)
- falsy values: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`
- `true` → `1`, `false` → `0`
- `==` : performs type conversion,
  `===` : also checks for type equality
- `-` operator: always treats operands as numbers

Conditionals

- if - else if - else

```
if (x > 0) {
  console.log("x pos");
} else if (x < 0) {
  console.log("x neg");
} else {
  console.log("x == 0");
}
```

- Ternary operator

```
let result = x % 2 === 0
  ? "even"
  : "odd";
```

- switch - case - default

```
switch (x) {
  case 1:
    console.log("x == 1");
    break;
  case 2:
    console.log("x == 2");
    break;
  default:
    console.log("x is neither");
}
```

Loops

- for loop

```
for (let i = 0; i < 10; i++) {
  console.log(i);
}
```

- while loop

```
while (i < 10) {
  ...
}
```

- do while loop

```
do {
  ...
} while (i < 10);
```

- for...in loop

```
const obj = { a: 1, b: 2, c: 3 };
for (const key in obj) {
  console.log(key, obj[key]);
}
```

- for...of loop

```
const arr = [1, 2, 3];
for (const value of arr) {
  console.log(value);
}
```

Functions

- `function` keyword

```
function add(a, b) {
  return a + b;
}
```

- arrow function (lambda)

```
const add = (a, b) => a + b;
```

- multi-line function expression

```
const add = function(a, b) {
  return a + b;
};
```

All of them are called in the same way, e.g.: `add(1, 2)`.

Arrays

- They can contain an arbitrary (dynamic) number and types of elements:

```
let array = [false, 1, "two", [0, 1, 2]];
```

- Zero-based indexing:

```
console.log(array[0]);    // false
console.log(array[3][0]); // 0
```

- They store their own length: `array.length`
- At the end
  - insert: `array.push(elem);`
  - remove: `array.pop();`
- At the beginning
  - insert: `array.unshift(elem);`
  - remove: `array.shift();`

Array functions

Let `l` be an arbitrary logical expression on the elements of array `array`. The array class (`Array`) implements a number of additional methods related to common algorithmic patterns:

- `find()` - search

  ```
  array.find((elem) => l); // one element of array
  ```

- `filter()` - filtering

  ```
  array.filter((elem) => l); // array built from elements
  ```

  its result is another array ⇒ you can call further `Array` methods on it

- `every()` and `some()` - universal and existential quantifiers

  ```
  array.every((elem) => l); // true for all?
  array.some((elem) => l);  // true for at least one?
  ```

Array functions

- `forEach()` - a function-level `for-each` loop
  ```
  array.forEach((elem) => console.log(elem));
  ```
- `flat()` - converting a multidimensional array into a contiguous (flat) array
  ```
  [[1, 2], [3, 4, [5]]].flat(); // [1, 2, 3, 4, 5]
  ```
- `join()` - concatenating the elements of an array into a string
  ```
  [1, 2, 3, 4].join(", "); // "1, 2, 3, 4"
  ```
- `map()` - transforming the elements of an array
  ```
  [1, 2, 3, 4].map((elem) => elem * 2); // [2, 4, 6, 8]
  [{a: 1, b: 2}, {a: 3, b: 4}].map((e) => e.a); // [1, 3]
  ```
- `reverse()` - reversing the elements of an array
  ```
  [1, 2, 3, 4].reverse(); // [4, 3, 2, 1]
  ```

Array functions

With the reducer function (`reduce()`) you can express further algorithmic patterns:

- summation

```
array.reduce((acc, curr) => acc + curr, 0);
```

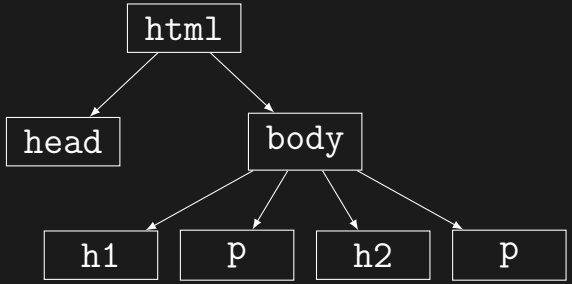
- maximum selection

```
array
  ? array.reduce((acc, curr) => Math.max(acc, curr))
  : null;
// or
Math.max(...array); // ... = spread operator
```

DOM - Document Object Model

The **D**ocument **O**bject **M**odel is the representation of the page's HTML code, in which every element has a corresponding JavaScript object.

```
<!DOCTYPE html>
<html>
  <head></head>
  <body>
    <h1>Title</h1>
    <p>Text</p>
    <h2>Subtitle</h2>
    <p>Even more text</p>
  </body>
</html>
```



DOM - Accessing elements

To select elements, you can use CSS selectors.

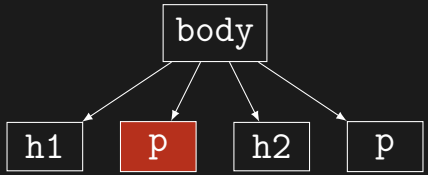
The first matching element can be queried with

```
document.querySelector('p');
```

If there are no matching elements, then it returns `null`.

The matching of an element to a CSS selector can be checked with

```
elem.matches('p'); // true or false
```



DOM - Accessing elements

All matching elements can be queried with

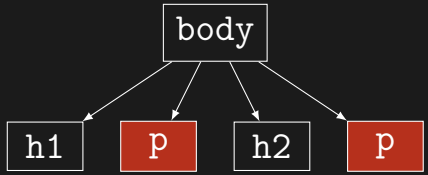
```
document.querySelectorAll('p');
```

This returns a `NodeList` object, which can be iterated with a

```
for (let elem of allElems)
```

It can be converted into an `Array` with

```
Array.from(allElems);
```



DOM - CSS selectors

- Basic selectors:
  - Type: `div`, `span`
  - Class: `.classname`
  - ID: `#id`
  - Filtered type: `div.classname`, `span#id`
  - Attribute: `div[attrib=value]`
  - Pseudo-class: `div:hover`, `div:visited`, etc.
  - Pseudo-element: `div::before`, `div::after`
- Combined selectors
  - Descendant: `div span`
  - Child: `div > span`
  - Subsequent sibling: `div` $\sim$ `span`
  - Adjacent sibling: `div + span`

Source: *MDN Web Docs*

DOM - Accessing attributes

Let our HTML tag be an

```
<input value="42" data-id="1" readonly>
```

input field, which we can query and store with

```
const input = document.querySelector('input');
```

Through the `input` variable we can read the element's attributes

```
input.readOnly; // true
input.dataset.id; // "1"
```

and we can also write them

```
input.value = "1337";
```

DOM - Accessing CSS attributes

Let our HTML tag be an

```
<input class="bg-red td-bold" style="color: blue;">
```

in a variable named `input`.

Through the `input` variable we can read and write the element's CSS attributes

```
input.style.color = "red";
```

and manipulate its CSS classes with

```
input.classList.add("bg-green");
input.classList.contains("bg-green"); // false
input.classList.toggle("bg-green"); // add or rem.
input.classList.remove("bg-red");
```

DOM - Accessing content

Let our HTML tag be a

```
<span id="test">Text</span>
```

piece of text, which we can query and store with

```
const span = document.querySelector('#test');
```

Through the span variable we can read its inner content

```
span.innerHTML; // "Text"
```

and we can also write it

```
span.innerHTML = "More text";
```

DOM - Adding and removing elements

The DOM representation of an element can be created with

```
const elem = document.createElement("span");
elem.textContext = "Hello world!";
```

The created element can be added

```
document.body.appendChild(elem);
```

and later removed from the DOM with

```
elem.remove();
```

Events

Events follow a **WHEN-THEN** logic, also known from user stories. When (**WHEN**) a certain event happens, then (**THEN**) a certain function is executed.

```
function handler() { /* THEN */ }
elem.addEventListener("type", handler); // WHEN
```

Events - Delegation

Let our HTML tags be the

```
<div> <input/><input/><input/> </div>
```

input fields. We store their parent with

```
const div = document.querySelector("div");
```

When the same event handler needs to be assigned to many similar descendants of an element, we use delegation. In this case the event handler is attached to the parent element.

```
delegate(div, "click", "input", handler);
    // parent,    WHEN,  child,     THEN
```

Events - Event type

```
elem.addEventListener("type", (event) => { /* THEN */ });
```

- `MouseEvent` - mouse event
  - `"click"` - mouse click
  - `"mousemove"` - mouse movement
  - `"mousedown"`, `"mouseup"` - pressing and releasing a mouse button
  - `"scroll"` - scrolling on the page
- `KeyboardEvent` - keyboard event
  - `"keypress"` - pressing a key on the keyboard
  - `"keydown"`, `"keyup"` - pressing and releasing a key on the keyboard
- `InputEvent` - input event
  - `"input"` - change of the value of an input field
- `SubmitEvent` - submit event
  - `"submit"` - sending a form

Events - Event object

```
elem.addEventListener("type", (event) => { /* THEN */ });
```

- `Event` - event
  - `type` - the type of the event (e.g. `click`)
  - `target` - the target of the event (can even be a child)
  - `preventDefault()` - prevents the default behaviour
  - `stopPropagation()` - prevents bubbling upon delegation
- `MouseEvent` - mouse event
  - `clientX`, `clientY` - the X and Y coordinates of the click on the page
  - `button` - the button(s) of the click (0 = left, 1 = wheel, 2 = right, etc.)
- `KeyboardEvent` - keyboard event
  - `key`, `code` - the pressed key (e.g. `"w"`, `"KeyW"` or `"1"`, `"Digit1"`)
- `InputEvent` - input event
  - `data` - the text that was typed or pasted
  - `inputType` - the type of change (e.g. `"insertText"`, `"insertFromPaste"`, `"deleteContentBackward"`)